

Roteiro de Falas da Apresentação

Análise comparativa de formulações de Fluxo de Potência Ótimo com ações de controle em JuMP/Julia

Gabriel Rufino Montenegro • Orientador: Prof. Lucas Silveira Melo • EE/UFC

Como usar este roteiro. Cada bloco abaixo corresponde a um slide da apresentação, na mesma ordem. A faixa de tempo à direita é o *tempo acumulado* ao *final* daquele slide — use um relógio/cronômetro como guia. As marcações em *[cinza itálico]* são deixas cênicas (não devem ser lidas). O texto em preto é a fala propriamente dita; não precisa ser decorada palavra por palavra, mas o ritmo foi calibrado para fechar em torno de **22 minutos** (janela de 20 a 25). Se perceber que está atrasado, os slides de Fundamentação (Fluxo de Potência AC e Ótimo) são os mais comprimíveis.

Resumo do tempo: Abertura 1,5 min → Fundamentação ~5 min → Metodologia ~4 min → Resultados ~8 min → Conclusões e encerramento ~3 min.

Bloco 1 — Abertura

Slide 1. Capa

0:00 – 0:45

Bom dia a todos. Cumprimento os membros da banca — professor Raimundo Furtado Sampaio, professor Rafael Castro de Andrade e o engenheiro Mário Sérgio, da Intertechne —, o meu orientador, professor Lucas Silveira Melo, e os demais presentes. Meu nome é Gabriel Rufino Montenegro e vou apresentar meu Trabalho de Conclusão de Curso, intitulado “Análise comparativa de formulações de Fluxo de Potência Ótimo com ações de controle em JuMP/Julia”. Em uma frase: eu desenvolvi, desde a concepção, uma sequência de seis formulações de fluxo de potência que incorporam, uma a uma, as ações de controle que o operador brasileiro usa na prática, e as testei desde redes de três barras até um cenário completo do Sistema Interligado Nacional.

Slide 2. Sumário

0:45 – 1:15

A apresentação tem quatro blocos. Primeiro, a motivação e os objetivos — por que esse problema importa. Depois, a fundamentação teórica: o que é fluxo de potência, fluxo de potência ótimo e quais são as ações de controle. Em seguida, a metodologia, com a arquitetura de software em Julia e a descrição das seis formulações. E, por fim, os resultados sobre o conjunto de casos de teste e as conclusões. *[avançar para o próximo slide]*

Bloco 2 — Motivação e Objetivos (~2:45 de bloco)

Slide 3. Contexto: operação do SIN

1:15 – 2:45

A operação de um sistema elétrico de grande porte, como o nosso Sistema Interligado Nacional, não se resume a resolver as equações de fluxo de potência. Ela depende de um conjunto de **ações de controle** que mantêm o sistema dentro de limites seguros. São cinco que me interessam aqui: o **QLIM**, que são os limites de geração de potência reativa dos geradores; o **VLIM**, os limites de magnitude de tensão nas barras; o **CSCA**, o chaveamento automático de bancos *shunt*, capacitores e reatores; o **CTAP**, o ajuste automático dos *taps* dos transformadores; e o que eu chamo de **DERA**, um esquema hierárquico de corte e alívio de carga em emergência. No Brasil, essas ações estão codificadas no programa ANAREDE, do CEPEL, e nos arquivos de dados no formato PWF.

O ANAREDE as resolve por meio de heurísticas e laços iterativos próprios. Sem representar esses controles, um ponto de operação calculado em computador pode parecer válido e, ainda assim, estar longe do que o sistema realmente faria — por exemplo, ignorando que um gerador já saturou seu suporte de reativo, ou que um banco de capacitores precisaria ter sido chaveado. A pergunta que move o trabalho é: *como reproduzir esses controles em uma ferramenta moderna, aberta e transparente?*

Slide 4. Lacuna metodológica

2:45 – 4:00

E aqui aparece a lacuna. Do lado acadêmico, a referência internacional é o pacote *PowerModels.jl*, escrito em Julia. Ele é excelente: oferece o fluxo de potência ótimo AC e suas relaxações. Mas ele *não* tem suporte nativo às ações de controle do ANAREDE. Havia uma proposta de preencher essa lacuna, o pacote *ControlPowerFlow.jl*; porém, ao longo do trabalho, eu confirmei, em contato com os desenvolvedores dos pacotes PWF.jl e ControlPowerFlow, que a versão pública ficou incompleta após uma parceria com uma empresa. Ou seja, não dava para usá-lo como referência de comparação. Diante disso, minha estratégia foi desenvolver as formulações **desde a concepção**, em JuMP, adicionando os controles um a um, de modo que cada uma pudesse ser comparada contra as duas referências do PowerModels sobre o mesmo conjunto de casos de teste.

Slide 5. Objetivos

4:00 – 5:00

O objetivo geral, portanto, foi construir, validar e comparar essa progressão incremental de formulações em JuMP/Julia, sobre o *solver* Ipopt, incorporando as ações de controle do ANAREDE. Entre os objetivos específicos, destaco quatro: documentar a formulação matemática do fluxo de potência em coordenadas polares; usar o pacote *PWF.jl* para ler os arquivos ANAREDE com as estruturas de controle; implementar duas formulações de referência com o PowerModels e quatro formulações próprias em JuMP; e, por fim, submeter as seis a um conjunto padronizado de vinte e sete casos de teste, do três barras ao SIN.

Bloco 3 — Fundamentação Teórica (~4:30 de bloco)

Slide 6. Fluxo de Potência AC

5:00 – 6:15

Rapidamente, a base teórica. O fluxo de potência AC é o conjunto de equações de balanço de potência ativa e reativa em cada barra: o que entra de geração, menos o que sai de carga, tem que fechar com o que circula pelos ramos. As barras são classificadas em três tipos: a *slack*, que é a referência de ângulo; as PV, com tensão e potência ativa especificadas; e as PQ, com as potências dadas e tensão e ângulo como incógnitas. O ponto importante para o restante da apresentação é que esse sistema é **não linear e não convexo** — ele é tradicionalmente resolvido por Newton-Raphson, e a não convexidade é justamente o que torna o problema difícil em larga escala.

Slide 7. Fluxo de Potência Ótimo e relaxações

6:15 – 7:45

Quando a esse sistema eu adiciono uma função objetivo — tipicamente o custo de geração — e o submeto a restrições, eu tenho o fluxo de potência *ótimo*. Lendo a equação do slide: eu minimizo o custo total de geração, *sujeito a* — é o que significa o “s.a.” — que o balanço de potência feche e que os limites operacionais sejam respeitados; esses limites incluem as faixas de tensão, os limites de geração e o limite térmico das linhas. Uma observação que retomo mais à frente: como no meu caso não há dados reais de custo das fontes, minimizar custo com coeficientes iguais equivale a minimizar a geração total e, pela conservação de energia com carga fixa, a minimizar as perdas. Como ele herda a não convexidade, surgiram na literatura várias relaxações. Há uma distinção

conceitual que vale frisar: a aproximação **DC** lineariza as equações — ela é rápida, mas *não* dá garantia sobre o ótimo verdadeiro; já as relaxações **SOC** e **SDP** são, por construção, garantidas: fornecem uma cota inferior *válida* do custo real. A tabela mostra isso num caso de cinco barras: o DC fica *abaixo* do AC porque ignora perdas e impõe condições irreais; e o SOC fica ainda mais abaixo, com um *gap* de cerca de dezoito por cento, o que é esperado em redes malhadas. Esse estudo é apenas didático — a progressão principal do trabalho mantém a formulação AC exata.

Slide 8. Ações de controle do ANAREDE

7:45 – 9:30

Este é o slide conceitualmente central. Eu represento cada controle assim: o **QLIM** e o **VLIM** reproduzem a lógica de troca de tipo de barra, PV para PQ, mas em vez de chavear de forma descontínua — o que atrapalha um *solver* suave como o Ipopt —, eu uso **variáveis de folga penalizadas** na função objetivo. O **CSCA** transforma a susceptância do banco *shunt* numa variável contínua de decisão, dentro de seus limites. O **CTAP** faz o mesmo com a razão de *tap* do transformador, e reaproveita as folgas de tensão já existentes, sem criar folga nova. E o **DERA** é um corte hierárquico de carga: o *solver* pode cortar parte da carga ativa de cada barra, mas com penalidade crescente conforme o nível de tensão — ele corta primeiro onde é mais fácil religar e protege a extra-alta tensão. O fio condutor é que *todos* os controles entram como *soft-constraint*, ou seja, restrições flexíveis penalizadas, evitando os laços descontínuos do ANAREDE. E cada folga tem significado físico claro: a folga de tensão do QLIM, por exemplo, corresponde ao gerador saturando o regulador automático de tensão e travando no limite de capacidade — a tensão da barra só se afasta do *setpoint* quando isso acontece, que é exatamente a conversão de barra PV para PQ que o ANAREDE executa no seu laço de troca de tipo. Já a folga de reativo do VLIM permite que a demanda reativa da barra de carga se ajuste minimamente para manter a tensão dentro dos limites.

Bloco 4 — Metodologia (~3:45 de bloco)

Slide 9. Fluxo de execução

9:30 – 10:45

Do ponto de vista de software, tudo foi feito em Julia, encadeando quatro pacotes: o *PWF.jl* para ler o arquivo ANAREDE, o *PowerModels.jl* como referência, o *JuMP.jl* para escrever os modelos e o *Ipopt* como *solver* não linear. O fluxo de execução tem seis etapas, sempre as mesmas: leitura do PWF já com os dados de controle; uma adequação da topologia que mantém só a maior componente conexa e uma única barra de referência; construção da estrutura de referência; montagem do modelo no JuMP; resolução; e exportação dos resultados em CSV. Um detalhe de modelagem: os elos de corrente contínua, os HVDC, têm suas injeções **fixadas** como condição de contorno — elas já vêm do fluxo base, e liberá-las só aumentaria o problema sem trazer informação.

Slide 10. A progressão F0 → F5

10:45 – 12:00

Esta tabela resume as seis formulações. A **F0** é o fluxo de potência ótimo de referência, do PowerModels; a **F1** é o fluxo de potência clássico, também do PowerModels. Daí em diante começam as minhas formulações em JuMP: a **F2** acrescenta QLIM e VLIM; a **F3** acrescenta o CSCA; a **F4**, o CTAP; e a **F5**, o DERA. O ponto-chave é que a progressão é **estritamente aditiva**: cada formulação parte da anterior e acrescenta um único bloco de variáveis e restrições. Isso tem uma consequência metodológica forte — se duas formulações consecutivas divergem num resultado, essa divergência é atribuível *exclusivamente* ao novo controle introduzido. É o que me permite isolar o efeito de cada controle.

Slide 11. Como funcionam as folgas penalizadas**12:00 – 13:15**

Para deixar concreto, veja a F2. As variáveis primárias são as de sempre — tensão, ângulo, potências —, com os limites lidos do PWF. A novidade são as folgas: uma folga de tensão em cada barra de geração — o mecanismo do QLIM — e uma folga de reativo em cada carga — o mecanismo do VLIM. A restrição de *setpoint* diz que a tensão da barra é o valor de referência mais a folga; e a função objetivo minimiza a soma dos quadrados dessas folgas, com um peso alto, de um milhão. A intuição é simples e é o coração do trabalho: *se* o problema é factível respeitando os *setpoints*, a folga fica zero e a solução respeita tudo. *Se não é*, em vez de o *solver* simplesmente declarar “infactível”, como faz o fluxo clássico, ele encontra a solução que **viola minimamente** os controles. É essa robustez que vai aparecer nos resultados.

Bloco 5 — Resultados (~8:00 de bloco)**Slide 12. Conjunto de casos de teste — 27 casos****13:15 – 14:15**

O conjunto tem vinte e sete casos de teste, escolhidos para varrer dois eixos: o porte da rede e a presença de cada controle. Há redes pedagógicas de três a nove barras, com variantes que ativam cada seção do PWF; redes de médio porte de trezentas e quinhentas barras; dois recortes reduzidos do Nordeste do SIN, de mil e de dois mil e seiscentas barras, derivados de um caso real do ciclo de planejamento do ONS; o cenário integral do SIN, com mais de treze mil barras; e alguns casos sintéticos de estresse. O critério para dizer que duas formulações chegaram à “mesma” solução é rígido: diferença menor que dez à menos quatro, simultaneamente, em tensão, ângulo, potências e fluxos.

Slide 13. Convergência global**14:15 – 15:45**

E este é o primeiro resultado central. Olhando a faixa de convergência no conjunto completo de casos: minhas formulações desenvolvidas manualmente, da F2 à F5, convergem em **vinte e três** dos vinte e sete casos. As referências convergem em menos: dezoito para a F1 e quinze para a F0. A tabela colorida mostra uma amostra — repare nas linhas em amarelo, como o `4busfrank_vlim`, o `5busfrank_csca` e o `caso_red2`: ali as referências declaram infactibilidade, e as formulações controladas convergem. No total, há **nove** casos em que pelo menos uma referência falha e pelo menos uma das minhas formulações encontra solução. A razão é exatamente aquela das folgas: onde o limite operacional rígido inviabiliza o problema, a folga penalizada recupera um ponto de operação utilizável. E quero ser preciso sobre o que isso significa: não é que minhas formulações sejam “melhores” por convergir mais; é que elas respondem a uma pergunta diferente — em vez de apenas dizer “infactível”, elas dizem *o quanto* e *onde* o sistema precisaria violar os controles para conseguir operar.

Slide 14. Análise barra a barra — clusters**15:45 – 17:00**

Convergir é uma coisa; *para onde* cada formulação converge é outra. Aqui eu agrupo as soluções em *clusters* de equivalência. Veja a rede de três barras: aparecem quatro soluções distintas. A **F0**, o ótimo do FPO — que, na ausência de dados reais de custo das fontes, equivale na prática a minimizar as perdas ativas —, empurra a tensão para o limite de 1,1 por unidade. A **F1** dá o ponto operacional “natural” da rede. A **F2** fica perto da F1, com uma folguinha residual. E as **F3 a F5**, que têm o CSCA ativo, deslocam o ponto para tensões consistentemente mais baixas, porque o controle do banco *shunt* ajusta a susceptância para perseguir o *setpoint*. Ou seja, cada *cluster* é uma assinatura computacional de um controle — o que serve, inclusive, para detectar regressões em implementações futuras.

Slide 15. Destaque: o caso_red2**17:00 – 18:20**

Esse caso é o exemplo mais nítido de todo o conjunto de casos de teste, e eu gosto de detê-lo aqui. É um recorte real do Nordeste, com mil e trinta e três barras. O fluxo de potência clássico, a F1, declara infactibilidade — mas repare *como*: ele entrega uma “solução” com tensão mínima de **menos zero vírgula quatro** por unidade e máxima de quase dois por unidade. Isso é fisicamente impossível. Já as formulações controladas, da F2 à F5, convergem para tensões dentro de uma faixa realista. E mais: a F4, que tem CSCA mais CTAP, reduz a função objetivo a cerca de dez à menos dois — algo como cinco mil vezes menor que a F2. Na prática, a combinação dos dois controles encontra um ponto de operação praticamente compatível com os *setpoints* do arquivo, quase sem violação. É a demonstração concreta de para que servem essas formulações.

Slide 16. O caso SIN integral**18:20 – 19:35**

Agora, o caso de maior interesse prático: o SIN integral, com treze mil trezentas e trinta e oito barras. Aqui eu preciso ser honesto sobre um resultado *negativo*: **nenhuma** das seis formulações converge dentro do orçamento de três mil iterações e do tempo limite. A F0, a F2 e a F4 estouram o tempo; a F1, a F3 e a F5 terminam em infactibilidade local. Mas há uma leitura importante junto: o sistema *não quebra* — toda a infraestrutura, leitura do PWF, construção da referência, montagem do modelo e chamada ao Ipopt, funciona corretamente em escala SIN. Não há erro de *parser* nem *crash*. E um ponto importante: minhas formulações *não* partem de uma inicialização trivial — elas já são inicializadas com o ponto de operação convergido pelo próprio ANAREDE, lido do arquivo PWF. Mesmo assim não fecham; e a F1, a referência do PowerModels, partindo do mesmo ponto, também termina infactível. Ou seja, o gargalo não está na inicialização: está na fidelidade do modelo reconstruído — eles HVDC, intercâmbios de área, controles remotos, múltiplas barras de referência — e no condicionamento numérico do problema nessa escala. O obstáculo é **numérico**, não estrutural.

Slide 17. Verificação auxiliar FDC**19:35 – 20:50**

E para comprovar essa hipótese eu fiz uma verificação adicional, que chamo de FDC: resolver o *mesmo* arquivo do SIN, mas na aproximação DC, linear. O resultado é o oposto: ele converge em poucas iterações, com status resolvido. A tabela traz os números agregados — treze mil barras retidas, cerca de setenta e um gigawatts de geração, perdas nulas por definição. Isso **confirma** que o obstáculo do AC é numérico, não estrutural: a mesma rede, no mesmo fluxo de execução, fecha quando linearizada. Há um porém honesto: os ângulos da solução DC chegam a cento e setenta graus, o que viola as próprias hipóteses da linearização — então essa solução não é fisicamente realizável e, com tensão um por unidade em todas as barras, ela seria um ponto de partida *pior* que o ponto convergido do ANAREDE que eu já uso. Por isso o valor da FDC é *diagnóstico*: ela confirma que existe solução para a topologia e que o obstáculo do AC é numérico — o caminho de trabalho futuro é reconciliar o modelo com o do ANAREDE e melhorar o condicionamento, não trocar o ponto de partida.

Bloco 6 — Conclusões e encerramento (~3:00 de bloco)**Slide 18. Síntese das contribuições****20:50 – 22:20**

Em síntese, três contribuições. A primeira: as formulações controladas ampliam a faixa de convergência — vinte e três de vinte e sete casos, recuperando soluções factíveis onde as referências falham. A segunda: os *clusters* de equivalência funcionam como uma “impressão digital” computacional de cada controle, reproduzindo o comportamento esperado do QLIM, do CSCA, do CTAP e

do DERA. E a terceira, que é uma contribuição negativa mas relevante: eu delimitei com clareza a fronteira atual — nenhuma formulação converge no SIN, mesmo partindo do ponto já convergido pelo ANAREDE, o que mostra exatamente onde a abordagem *soft-constraint* pura precisa de reconciliação de modelo e de melhor condicionamento numérico. Em conjunto, essas três contribuições mostram que a abordagem é robusta nas escalas pedagógica e regional, e mapeiam, com precisão, o que ainda falta para alcançar a escala nacional.

Slide 19. Limitações e trabalhos futuros

22:20 – 23:35

As principais limitações: o CSCA e o CTAP são tratados como variáveis contínuas, enquanto a operação real é discreta; o DERA é uma versão contínua e suave de um mecanismo que, na realidade, é discreto e emergencial; e os elos HVDC entram com fluxos fixados. Os trabalhos futuros decorrem diretamente disso: primeiro, viabilizar o SIN atacando a causa efetiva — uma verificação de resíduos alimentando o modelo com a solução do PWF, uma auditoria da tradução PWF para JuMP e o escalonamento progressivo das penalidades por homotopia; segundo, discretizar o CSCA e o CTAP, transformando o problema num MINLP, com os *solvers* que o ecossistema Julia já oferece; e, terceiro, acoplar as minhas formulações ao *ControlPowerFlow.jl* como ferramenta de validação.

Slide 20. Encerramento

23:35 – 24:05

Com isso eu encerro. O trabalho mostrou que é possível reproduzir, de forma transparente e reprodutível, as ações de controle do ANAREDE em uma ferramenta aberta, e mapeou tanto os ganhos quanto a fronteira numérica atual da abordagem. Agradeço novamente à banca pela atenção e ao professor Lucas pela orientação, e fico à disposição para as perguntas. Muito obrigado.

Apêndice — Perguntas prováveis da banca (preparação)

- 1. Por que penalidade quadrática e por que $\rho = 10^6$?** O quadrado mantém a função objetivo suave e diferenciável, ideal para o Ipopt; o valor 10^6 é alto o bastante para que, havendo solução sem violação, as folgas saturem em zero, mas não tão alto a ponto de degradar o condicionamento numérico. Na F5 eu reduzo para 10^5 justamente para que o termo de corte de carga se torne competitivo.
- 2. Tratar CSCA/CTAP como contínuos não descaracteriza o controle?** Captura corretamente a *direção* e a *magnitude* da compensação que o operador faria; perde-se só a granularidade discreta dos passos. A discretização via MINLP é o passo seguinte, e está nos trabalhos futuros.
- 3. Por que o SIN não converge se os recortes convergem?** Não é a inicialização: todas as formulações já partem do ponto convergido pelo ANAREDE, lido do PWF — e mesmo a F1, do PowerModels, partindo desse ponto, termina infatível. A diferença está no que só o caso integral tem: elos HVDC, intercâmbios de área, controles remotos e múltiplas barras de referência, cuja tradução PWF→JuMP precisa ser auditada, além do condicionamento do NLP com penalidade 10^6 nessa escala. A FDC confirma que o obstáculo é numérico, não estrutural — por isso as frentes são verificação de resíduos, auditoria da tradução e homotopia das penalidades.
- 4. Qual a diferença essencial para o ANAREDE?** O ANAREDE resolve por laços iterativos externos e chaveamentos discretos; eu resolvo tudo numa única otimização não linear com folgas. Os resultados convergem para o mesmo tipo de ponto de operação, mas com uma formulação explícita e auditável.
- 5. O que é exatamente o DERA neste trabalho?** É um rótulo herdado do nome do *script*. Aqui ele designa um esquema hierárquico de corte de carga, análogo aos Esquemas Regionais de

Alívio de Carga do ONS, em que o *solver* corta carga preferencialmente nas barras de menor nível de tensão.

6. Por que Julia e não Python/MATLAB? Pela combinação de sintaxe de alto nível com desempenho de linguagem compilada, e porque o JuMP e o PowerModels.jl, referências em otimização de sistemas de potência, são nativos do ecossistema Julia.

7. Por que a F0 minimiza perdas se o objetivo é custo? Pela conservação de energia no sistema: geração total = carga total + perdas. A carga é dado fixo de entrada, então geração e perdas diferem apenas por uma constante — minimizar uma minimiza a outra. Como eu não tenho dados reais de custo das fontes e atribuo coeficientes iguais a todos os geradores, minimizar o custo total vira minimizar a geração total, que por sua vez é minimizar as perdas. Por isso o resultado da F0 deve ser lido como um ponto de mínimas perdas, não de mínimo custo econômico — e é essa a ressalva metodológica que registro na dissertação.

8. Na equação de balanço, por que o termo *shunt* é V_i^2 e não $V_i V_j$? Porque são elementos diferentes. O $g_i^{sh} V_i^2$ é um *shunt* ligado entre a barra i e o *terra* — como o outro lado é o terra, com tensão zero, só entra V_i , e o subíndice é i porque o elemento está preso a uma barra só. O acoplamento $V_i V_j$ que se espera existe, sim, mas dentro do fluxo de ramo p_{ij} : ao abrir esse termo aparecem tanto um termo próprio V_i^2 quanto o termo mútuo $V_i V_j$, agora com as admitâncias de subíndice ij , porque aí o elemento é a linha entre as duas barras.

— Perguntas específicas de otimização —

9. Qual o método de otimização utilizado? O *solver* é o Ipopt, que implementa um método de **pontos interiores primal-dual** com busca em linha do tipo *filter* (Wächter e Biegler, 2006). A ideia central: as restrições de desigualdade (os limites) são incorporadas à função objetivo como termos de *barreira logarítmica* $-\mu \sum \ln(\text{folga})$, que impedem as iterações de tocar as fronteiras; o *solver* resolve uma sequência de subproblemas, reduzindo o parâmetro de barreira $\mu \rightarrow 0$, e a cada passo aplica o método de Newton sobre o sistema de condições de otimalidade (KKT) perturbadas, o que envolve resolver um sistema linear esparso. É por exigir derivadas de primeira e segunda ordem que o método precisa de funções *suaves* — contínuas e diferenciáveis —, e é isso que motiva toda a estratégia de folgas em vez de lógicas discretas.

10. O problema é convexo? Como garantir o ótimo global? Não — o FPO AC é **não convexo**, por causa dos termos $V_i V_j \cos(\theta_i - \theta_j)$. O Ipopt, por ser um método local, encontra apenas um *ponto KKT local*, sem garantia de ótimo global. Eu mitigo isso inicializando as formulações no ponto de operação já convergido pelo ANAREDE, que é fisicamente sensato. Garantia global exigiria *branch-and-bound* espacial ou o uso de relaxações convexas (SOC, SDP) para fornecer uma cota inferior e limitar o *gap* de otimalidade — foi justamente o que o estudo do `case5.m` ilustrou.

11. Como o Ipopt trata os limites de desigualdade? Pelo termo de barreira: em vez de testar quais restrições estão ativas (como um método de conjunto ativo ou o *simplex*), ele mantém as iterações estritamente no *interior* da região factível e deixa que a redução de μ as empurre suavemente contra os limites que forem ativos na solução. Por isso o nome “pontos interiores”.

12. O que significam os *status* `LOCALLY_SOLVED` e `LOCALLY_INFEASIBLE`? `LOCALLY_SOLVED` quer dizer que o Ipopt encontrou um ponto que satisfaz as condições KKT de primeira ordem dentro da tolerância — um ótimo *local*. `LOCALLY_INFEASIBLE` quer dizer que ele convergiu para um mínimo local da *violação* das restrições sem atingir factibilidade; é importante frisar que isso *não* prova que o problema é globalmente infactível — apenas que, a partir daquele ponto de partida, o *solver* não encontrou uma região factível.

13. Por que penalizar (*soft-constraint*) em vez de impor as restrições diretamente? Porque impor a tensão igual ao *setpoint* como igualdade rígida, *junto* com os limites rígidos de reativo, pode ser um sistema incompatível — sobredeterminado — e o *solver* devolve simplesmente INFEASIBLE. A penalidade relaxa o *setpoint* de forma controlada, garantindo um problema sempre

factível que se degrada suavemente: em vez de um modelo que “quebra”, tenho um que entrega a melhor solução possível com violação mínima. É essa a diferença que aparece nos casos em que só as minhas formulações convergem.

14. A penalidade quadrática é “exata”? As folgas realmente chegam a zero? No sentido técnico, não: penalidade *exata* exige uma norma não suave (tipo L_1), que o Ipopt não digere bem. Com a penalidade quadrática e ρ finito, as folgas tendem a zero apenas no limite $\rho \rightarrow \infty$; com $\rho = 10^6$ sobra um resíduo numericamente desprezível. Aumentar ρ reduz o resíduo mas piora o condicionamento — o escalonamento progressivo de ρ por homotopia é o caminho registrado nos trabalhos futuros.

15. Se discretizasse o CSCA e o CTAP, que classe de problema e que método? Viraria um MINLP — programação não linear inteira mista, com variáveis inteiras (o passo do banco, a posição do *tap*) convivendo com as equações não lineares da rede. A resolução típica é por *branch-and-bound*, que resolve uma sequência de relaxações contínuas — e as minhas F3 e F4 são exatamente essas relaxações. O ecossistema Julia já oferece *solvers* acoplados ao JuMP para isso: Juniper, Bonmin, Alpine ou SCIP. O custo computacional, porém, cresce muito.

16. Se a FDC (DC) converge sobre o SIN, por que a solução não presta — e por que não a uso como *warm-start*? Porque a aproximação DC só é válida sob hipóteses que essa própria solução viola. O DC lineariza o fluxo ativo trocando $\sin(\theta_i - \theta_j)$ por $(\theta_i - \theta_j)$, o que só vale para *diferenças angulares pequenas*. A troca é excelente a 10° (erro $\sim 0\%$), sofrível a 30° ($\sim 5\%$) e absurda a 170° : o transporte real, $\sin 170^\circ = 0,17$, é pequeno, mas o modelo linear usa 2,97 rad e superestima o fluxo em cerca de 17 vezes. É por isso que aparece um fluxo de ramo de 99,5 p.u., quase 10 GW numa linha — um valor que nenhuma linha real carregaria, pois passando de $\sim 90^\circ$ por linha o sistema síncrono já perde estabilidade estática. Então a solução DC é válida para o *modelo linearizado*, mas não corresponde a um ponto de operação fisicamente realizável da rede AC. Como ela ainda tem tensão 1,0 em todas as barras e ângulos de $\pm 177^\circ$, é um ponto de partida *pior* que o ponto já convergido do ANAREDE que eu utilizo — por isso o papel da FDC é **diagnóstico** (confirma que existe solução para a topologia, logo o obstáculo do AC é numérico, não estrutural), e não de inicialização. (*Se cutucarem: os $\pm 170^\circ$ são ângulos de barra em relação à slack, que acumulam ao longo de muitas linhas em série no SIN; a prova de que alguma linha está fora da faixa de pequeno ângulo é o próprio fluxo de 99,5 p.u., pois no DC $p_{ij} = \Delta\theta_{ij}/x_{ij}$.*)

17. O que quer dizer “o obstáculo do SIN é numérico, não estrutural”? São duas causas distintas para uma mesma falha de convergência. *Estrutural* significaria que a solução **não existe** — a rede seria genuinamente infactível, e aí nada adiantaria (nem inicialização, nem ajuste de *solver*). *Numérico* significa que a solução **existe, mas o algoritmo não chega até ela** por dificuldade de cálculo: mau condicionamento, escala, penalidades desequilibradas. A analogia é um labirinto: estrutural é um labirinto *sem saída*; numérico é um labirinto *com saída*, mas você está de olhos vendados e na neblina, e o tempo acaba antes. No meu caso, três evidências sustentam que é numérico: (i) o ANAREDE já achou um ponto de operação real dessa rede — então uma solução física existe; (ii) a FDC (DC) converge sobre a mesma topologia, logo há um modelo solúvel daquela rede; e (iii) mesmo **partindo do ponto bom**, todas as seis formulações e até a F1 do PowerModels falham — o que aponta para a fidelidade da tradução PWF→JuMP (elos HVDC, intercâmbios, controles remotos, múltiplas referências) e para o condicionamento do NLP com $\rho = 10^6$ em treze mil barras, não para a ausência de solução. Isso muda o desfecho do trabalho: em vez de um beco sem saída, o caminho à frente é engenharia numérica — verificação de resíduos, auditoria da tradução e homotopia de ρ —, não uma teoria nova.

18. Se a FDC não tem perdas, por que a geração ($\approx 70,97$ GW) difere da carga ($\approx 69,57$ GW)? Essa diferença de $\sim 1,4$ GW **não é perda** — em DC as perdas são exatamente zero por definição, e o balanço nodal completo fecha na igualdade. É um efeito de *contabilidade*: os elos HVDC do SIN (Itaipu, Madeira, Belo Monte) injetam e absorvem potência ativa real na rede AC, modelados como geradores fictícios de injeção fixada, mas essas injeções *não entram* na

coluna **P_Geracao** do CSV que eu uso para totalizar a geração — elas entram apenas no balanço interno do *PowerModels*. Somando ainda o pequeno ajuste absorvido pela(s) barra(s) de referência, quando se contabiliza a contribuição dos elos e da *slack* o balanço fecha exatamente. Ou seja, o 1,4 GW mede o que trafega pelos elos HVDC, não dissipação — é um artefato de qual coluna do CSV está sendo somada, não uma perda física.